SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	G.SANGHAMITRA	**Reg. No.:**	1924260351
Section / Batch:	D/ 2026	**Date:**	21/10/2026

Code of Conduct

I G.SANGHAMITRA(1924260351) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Sanghamitra

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Rubrics for Scenario Based Assignment

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Annexure A – Scenario Based Assignment

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.

- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption. • Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.

(b) Analyze the computational challenges of large game trees.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

(d) Design an evaluation function and justify the factors considered.

(e) Recommend a strategy for balancing optimality and real-time performance.

Q1. Emergency Medicine Delivery Drones in a Flood Zone

(a) Explain how Greedy Best-First Search behaves in this scenario. Discuss its strengths and weaknesses under uncertainty.

Answer:

Greedy Best-First Search is an informed search algorithm that selects the next node based on the heuristic value $h(n)$. In this scenario, the heuristic is the straight-line distance from the drone's current position to the destination.

- The drone chooses the path that appears closest to the destination.
- It can find a route quickly when the environment is simple.
- It reduces search time by prioritizing promising paths.
- However, it ignores the actual travel cost and focuses only on the estimated distance.
- It may select a flooded or blocked path because straight-line distance does not consider all real-world conditions.
- Therefore, it may produce a longer or unsafe route under uncertainty.

Conclusion: Greedy Best-First Search is fast but may not find the safest or least-cost route in a flood-affected region.

(b) Explain how A* handles the same situation, including how it balances cost and heuristic.

Answer:

A* is an informed search algorithm that considers both the actual cost of reaching a node and the estimated cost to reach the destination.

Its evaluation function is:

$$ f(n)=g(n)+h(n) $$

Where:

- $g(n)$ = actual cost from the starting point to the current node.
- $h(n)$ = estimated cost from the current node to the destination.
- $f(n)$ = estimated total cost of the path.

In this scenario, \(g(n)\) can include travel time, weather conditions, and terrain difficulty, while \(h(n)\) can represent straight-line distance converted into an estimated travel cost.

A* selects the path with the lowest estimated total cost. If a path becomes blocked, the drone can update the path information and search again.

Conclusion: A* generally provides a better balance between route efficiency and cost than Greedy Best-First Search. Its optimality depends on the heuristic and cost assumptions being satisfied.

(c) Analyze the impact of heuristic accuracy on both algorithms.

Answer:

A heuristic estimates how far a current state is from the goal. Its accuracy affects the efficiency and quality of the search.

- Greedy Best-First Search: It depends heavily on the heuristic. An inaccurate estimate may direct the drone towards a blocked or dangerous path.
- A Search:* It uses both actual cost and heuristic estimates. Therefore, actual travel costs can help compensate for an imperfect heuristic.
- An overestimated heuristic may cause A* to lose its optimality guarantee.
- An admissible heuristic never overestimates the true minimum remaining cost and helps A* find an optimal solution under the standard search assumptions.
- Inaccurate distance estimates can increase unnecessary exploration and delay delivery.

Conclusion: An accurate, suitable heuristic improves search efficiency and supports better route selection.

(d) Discuss how dynamic changes, such as blocked paths, affect search performance.

Answer:

Dynamic changes make the search problem more difficult because the drone's environment changes while it is travelling.

- Flooding may make previously available routes inaccessible.
- Weather changes may increase travel time or energy consumption.
- The drone must use real-time updates to identify blocked paths.

- Previously calculated routes may become invalid, requiring replanning.
- Repeated searches may increase computation time and delay medicine delivery.
- Frequent sensor updates and efficient replanning can improve safety and reliability.

Conclusion: The drone should continuously update its route information and replan whenever significant environmental changes occur.

(e) Recommend the most suitable algorithm and justify your decision.

Answer:

A* Search is the most suitable choice for this scenario because it considers both actual travel cost and estimated distance to the destination.

The reasons are:

1. Efficiency: It prioritizes paths with a low estimated total cost.
2. Optimality: With suitable non-negative costs and an admissible heuristic, standard A* can find an optimal route.
3. Safety: Terrain difficulty, weather, blocked paths, and risk can be included in the path cost.
4. Adaptability: The drone can update its map and replan when new information becomes available.
5. Reliability: It makes a more balanced decision than Greedy Best-First Search, which considers only the heuristic.

For strict real-time requirements, A* can be combined with efficient replanning techniques and safety checks.

Final recommendation: Use A* with real-time environmental updates and risk-aware path costs. If computation time becomes critical, use an incremental replanning method or a time-bounded search while prioritizing safe routes.

Q2. Smart City Traffic Signal Optimization

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

Answer:

Traffic signal optimization is an optimization problem in which the AI system finds suitable signal timings to reduce traffic congestion.

Objective: Minimize total waiting time, fuel consumption, and traffic congestion.

The problem can be formulated as follows:

- State: Current traffic conditions at all intersections.
- Decision variables: Green-light duration, red-light duration, and signal sequence.
- Objective function: Minimize waiting time, fuel consumption, and congestion.
- Constraints: Minimum and maximum signal timings, pedestrian crossing requirements, and traffic safety rules.

Traditional search methods may be inefficient because the number of possible signal combinations increases rapidly with the number of intersections. Traffic conditions also change continuously, making it difficult to evaluate every possible solution.

Conclusion: Local search and optimization algorithms are suitable for finding good solutions efficiently in a large search space.

(b) Explain how Hill Climbing works in this scenario and identify its limitations.

Answer:

Hill Climbing is a local search algorithm that starts with an initial solution and repeatedly moves to a better neighboring solution.

In traffic signal optimization:

1. The system begins with an initial signal timing plan.
2. It measures waiting time, fuel consumption, and congestion.
3. It changes the timing of one or more traffic signals.
4. It evaluates the new solution.
5. If the new solution is better, it accepts the change.
6. The process continues until no better neighboring solution is found.

Limitations:

- It can become trapped in a local optimum.
- It may stop on a plateau where neighboring solutions have similar values.
- It may struggle with ridges that require coordinated changes across multiple intersections.
- It does not guarantee the global optimum.
- Frequent traffic changes may require repeated optimization.

Conclusion: Hill Climbing is simple and fast but may not produce the best overall traffic signal plan.

(c) Discuss local maxima, plateaus, and ridges with real-world interpretation.

Answer:

These are common difficulties encountered by Hill Climbing.

1. Local maximum or local optimum

A solution is better than its nearby alternatives but is not the best solution overall.

Example: Changing the timing at one intersection reduces waiting time there, but a different timing plan across several intersections could reduce total city traffic.

2. Plateau

A plateau is an area in the search space where neighboring solutions have the same or nearly the same evaluation value.

Example: Several signal timing combinations produce almost identical waiting times. The algorithm may stop without finding a better solution.

3. Ridge

A ridge is a region where reaching a better solution may require coordinated changes in multiple variables.

Example: Two connected intersections may need their green-light timings adjusted together to create a smooth flow of traffic. Changing only one signal may not improve the result.

Conclusion: These problems can prevent Hill Climbing from finding a globally effective traffic signal configuration.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Answer:

Simulated Annealing and Genetic Algorithms are optimization techniques that can explore a large search space more effectively than basic Hill Climbing.

Simulated Annealing:

- It begins with an initial solution.
- It explores neighboring solutions.
- It sometimes accepts a worse solution to escape a local optimum.
- This acceptance is more likely at a high temperature and becomes less likely as the temperature decreases.
- It gradually focuses on improving the solution.

Example: The system may temporarily accept a signal plan with slightly higher waiting time to discover a better overall configuration.

Genetic Algorithm:

- It creates a population of possible signal timing plans.
- It evaluates each plan using a fitness function.
- It selects better-performing plans as parents.
- Crossover combines parts of different plans.
- Mutation introduces small changes to explore new possibilities.
- The process repeats over multiple generations.

Example: One plan may provide good timings for one group of intersections, while another performs well elsewhere. Crossover can combine these useful features.

Conclusion: Simulated Annealing can escape local optima by occasionally accepting worse solutions, while Genetic Algorithms explore different combinations through populations, crossover, and mutation.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Answer:

A Genetic Algorithm is a suitable choice for optimizing signal timings across hundreds of intersections because it can explore many candidate solutions simultaneously.

The reasons are:

1. Scalability: It can represent signal timings across multiple intersections in one candidate solution.

2. Global exploration: A population of solutions helps explore different regions of the search space.

3. Adaptability: The system can periodically update its population using new traffic information.

4. Flexibility: Multiple objectives, including waiting time, fuel consumption, and congestion, can be included in the fitness function.

5. Constraint handling: Traffic safety and signal timing restrictions can be enforced when generating or evaluating solutions.

However, Genetic Algorithms can require significant computation and parameter tuning. Simulated Annealing may be more suitable when a simpler optimization process is preferred.

Final recommendation: Use a Genetic Algorithm with real-time traffic data, safety constraints, and periodic optimization. For strict real-time operation, use a fast local adjustment mechanism between optimization cycles.

Q3. Autonomous Rover Exploring Mars

(a) Explain why this problem requires an online search agent instead of offline search.

Answer:

An online search agent makes decisions while interacting with an environment that is not completely known in advance. An offline search agent generally plans using a model or map available before execution.

The Mars rover requires online search because:

- It does not have a complete map of the terrain.

- Its sensors provide only partial information about the surroundings.

- New obstacles and hazards may be discovered during movement.
- It must make decisions using the information currently available.
- It needs to update its route as exploration continues.

Example: If the rover discovers a crater in its planned direction, it can update its knowledge and choose another route.

Conclusion: Online search is suitable because the rover must learn about its environment and make decisions while exploring.

(b) Analyze the challenges of partial observability and dynamic environments.

Answer:

Partial observability means that the rover cannot directly observe every detail of its environment. A dynamic environment changes while the rover is operating.

The main challenges are:

1. Limited sensing: The rover may not detect distant or hidden obstacles.
2. Unknown terrain: The surface may contain unobserved craters, rocks, or unstable ground.
3. Uncertainty: Sensor readings may be incomplete or inaccurate.
4. Navigation difficulties: The rover may need to change direction when new hazards are discovered.
5. Limited resources: Power, communication, and processing capabilities may be restricted.
6. Safety risks: Incorrect decisions may damage the rover or prevent sample collection.

Conclusion: The rover needs continuous sensing, cautious movement, and regular route updates to operate safely.

(c) Describe how the rover builds and updates its internal model.

Answer:

The rover builds an internal model by collecting sensor data and updating its representation of the surrounding terrain.

The process is as follows:

1. Initial knowledge: The rover begins with a limited map or basic information about the exploration area.

2. Sensing: It collects information using cameras and other available sensors.

3. Obstacle detection: It identifies rocks, craters, and other hazards.

4. Map updating: It adds newly observed information to its internal map.

5. State estimation: It estimates its current position and the condition of nearby terrain.

6. Path planning: It selects a suitable path based on the updated model.

7. Continuous feedback: It repeats sensing and updating as it moves.

Example: When the rover detects a large rock, it marks the area as an obstacle and calculates an alternative path.

Conclusion: Continuous model updating allows the rover to improve its understanding of unknown terrain and make informed decisions.

(d) Compare online search with uninformed and informed search strategies.

Answer:

Feature	**Online Search**	**Uninformed Search**	**Informed Search**
Knowledge	Learns about the environment during execution	Uses available problem information without a heuristic	Uses a heuristic to guide the search
Environment	Suitable for incompletely known environments	Usually searches a known problem representation	Usually searches a known problem representation
Guidance	Uses observations and feedback	Uses general rules such as breadth or depth	Uses heuristic estimates
Adaptability	Can respond to newly discovered information	Standard methods do not automatically adapt to new information	Standard methods do not automatically adapt to new information

Example	Rover exploring unknown terrain	Breadth-First Search	A* Search

Online search describes how an agent acts and learns during execution, while uninformed and informed search describe how search is guided. These categories are not completely exclusive: an online agent can also use heuristic information.

Conclusion: Online search is appropriate when the rover must discover the terrain while moving. Heuristics can further improve its route selection.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Answer:

An online search agent combined with heuristic-guided planning and continuous obstacle avoidance is the most suitable approach for the Mars rover.

The reasons are:

- Uncertainty: It can make decisions without requiring a complete map.
- Adaptability: It updates its internal model when new terrain information becomes available.
- Safety: It detects hazards and replans routes to avoid dangerous areas.
- Efficiency: Heuristics can help prioritize promising paths.
- Sample collection: It can plan routes that balance exploration, scientific objectives, and travel cost.

The rover should also use conservative movement rules and verify that a planned path is safe before execution.

Final recommendation: Use an online search agent with heuristic-guided planning, continuous sensing, map updates, and safety checks. This approach provides flexibility for exploring an unknown environment.

Q4. University Exam Timetable Generation

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP). Define the variables, domains, and constraints.

Answer:

A Constraint Satisfaction Problem (CSP) consists of variables, domains, and constraints. The goal is to assign a value to every variable while satisfying all constraints.

For university exam timetable generation, the CSP can be defined as follows.

1. Variables

Each exam is represented by a variable.

Example: \(E_1, E_2, E_3\) represent the exams for Mathematics, Physics, and Computer Science.

2. Domains

The domain of each variable contains the available exam slots and suitable halls.

Example: An exam may be assigned to Monday morning, Monday afternoon, or Tuesday morning, along with an available hall.

3. Constraints

- No overlapping exams: A student must not have two exams at the same time.
- Hall capacity: The assigned hall must have enough seats.
- Limited halls: The number of simultaneously scheduled exams cannot exceed available hall capacity.
- Precedence: Certain subjects must be scheduled before other subjects.
- Faculty availability: Exams must be scheduled according to faculty availability.
- Special accommodations: Students requiring special facilities must be assigned suitable halls and timings.

Example: If the same students attend Mathematics and Physics, the two exams cannot be scheduled in the same time slot.

Conclusion: The CSP model represents exams as variables, possible schedules as domains, and academic and logistical requirements as constraints.

(b) Explain how backtracking search works in this scenario.

Answer:

Backtracking search is a CSP-solving method that assigns values to variables one at a time and reverses assignments when a constraint is violated.

The steps are:

1. Select an unassigned exam.
2. Choose a possible time slot and hall from its domain.
3. Check whether the assignment satisfies all applicable constraints.
4. If valid, keep the assignment and select the next exam.
5. If no valid value remains, return to an earlier exam.
6. Change the previous assignment and try another value.
7. Continue until a complete valid timetable is found or all possibilities have been explored.

Example: Suppose Mathematics and Physics are assigned to the same slot, but some students attend both exams. The system rejects that assignment and tries a different slot for one of the exams.

Conclusion: Backtracking systematically searches for a valid timetable while avoiding assignments that violate constraints.

(c) Discuss how constraint propagation, such as forward checking, improves efficiency.

Answer:

Constraint propagation reduces the possible values of unassigned variables by using constraints discovered during the search.

Forward checking is a technique that checks the domains of related unassigned variables after each assignment.

The process works as follows:

1. Assign an exam to a time slot.
2. Identify other exams affected by that assignment.
3. Remove time slots that would violate constraints from their domains.
4. Check whether any unassigned exam has an empty domain.
5. If a domain becomes empty, backtrack immediately instead of continuing an impossible search.

Example: If Mathematics is scheduled for Monday morning, forward checking removes Monday morning from the domain of another exam that shares students with Mathematics.

Advantages:

- Detects conflicts early.
- Reduces the number of possibilities explored.
- Avoids unnecessary backtracking.
- Improves search efficiency as the number of exams increases.

Conclusion: Forward checking makes timetable generation more efficient by identifying impossible assignments before a complete timetable is constructed.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Answer:

Large universities face several challenges when generating exam timetables.

Challenges:

1. Thousands of students may be enrolled in different courses.
2. Multiple exams may share students or faculty.
3. The number of available halls is limited.
4. Hall capacities and accessibility requirements differ.
5. Some exams must occur before others.
6. Faculty availability and timetable changes may introduce new constraints.
7. The number of possible schedules increases rapidly as the number of exams grows.

Best strategy:

Use backtracking search combined with constraint propagation and suitable variable-ordering heuristics.

- Minimum Remaining Values (MRV): Select the exam with the fewest valid remaining slots first.

- Degree heuristic: Prefer exams that constrain the largest number of other unassigned exams when appropriate.
- Forward checking: Remove invalid options after each assignment.
- Constraint propagation: Apply additional consistency checks to reduce the search space.
- Incremental rescheduling: When a change occurs, revise the affected parts of the timetable where possible instead of rebuilding everything.

Conclusion: A CSP solver using backtracking, forward checking, and appropriate heuristics is a strong approach for generating valid timetables efficiently. For very large problems, specialized constraint solvers or optimization techniques may also be useful.

Q5. Strategic Game AI for Real-Time Decision Making

(a) Explain how the Minimax algorithm models this problem.

Answer:

Minimax is an adversarial search algorithm used in games where two players have opposing objectives. It assumes that both players choose moves to maximize their own chances of success.

In this scenario, the AI opponent attempts to maximize its advantage, while the human player attempts to reduce the AI's advantage.

The algorithm works as follows:

1. Represent the current game state as the root of a game tree.
2. Generate the possible moves from that state.
3. Expand the tree to a selected search depth.
4. Evaluate terminal or depth-limited states.
5. At a MAX node, choose the move with the highest evaluation.
6. At a MIN node, assume the opponent chooses the move with the lowest evaluation.
7. Select the best move based on the minimax values.

The minimax value can be expressed as:

$$ V(s)= \begin{cases} \max\limits_{a} V(s_a), & \text{MAX player}\\ \min\limits_{a} V(s_a), & \text{MIN player} \end{cases} $$

Here, s is the current state and s_a is the state reached after action a.

Conclusion: Minimax helps the AI choose moves by considering its own objectives and the likely responses of its opponent.

(b) Analyze the computational challenges of large game trees.

Answer:

A game tree represents the possible moves and resulting states in a game. In a complex real-time strategy game, the tree can become extremely large.

The main challenges are:

- Large branching factor: Each state may contain many possible moves.
- Increasing search depth: Looking further ahead creates many more states to evaluate.
- Time constraints: The AI must choose a move before its decision deadline.
- Memory requirements: Storing game states and search information can require substantial memory.
- Complex evaluation: Each state may involve resources, units, territory, and opponent behaviour.
- Changing game conditions: New actions and events may make earlier calculations less useful.

For a tree with branching factor b and search depth d, ordinary minimax has a worst-case time complexity of approximately:

$$ O(b^d) $$

Conclusion: Searching the entire game tree is generally impractical, so the AI needs pruning, depth limits, and efficient evaluation functions.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

Answer:

Alpha-Beta pruning is an optimization of Minimax that eliminates branches that cannot affect the final decision. It produces the same minimax result as Minimax when both algorithms search the same tree to the same depth.

It maintains two values:

- Alpha (α): The best value found so far for the MAX player.
- Beta (β): The best value found so far for the MIN player.

Pruning occurs when:

$$ \alpha \geq \beta $$

Working:

1. Start at the root and explore the game tree.
2. At MAX nodes, update alpha when a better value is found.
3. At MIN nodes, update beta when a lower value is found.
4. If alpha becomes greater than or equal to beta, stop exploring the remaining branches of that node.
5. Continue until the required search is complete and select the best move.

Example:

Suppose MAX has already found a move with value 5. While exploring another move, MIN finds a response with value 3. Since MAX already has an alternative worth 5, it will never choose the alternative whose value can be reduced to 3 or less. The remaining branches under that MIN node can be pruned.

Advantages:

- Reduces the number of nodes evaluated.
- Allows deeper searches within the same time limit.
- Reduces unnecessary computation.
- Preserves the minimax decision when applied correctly.

In the best case, with ideal move ordering, the time complexity can improve to approximately:

$$ O(b^{d/2}) $$

In the worst case, it remains $O(b^d)$.

Conclusion: Alpha-Beta pruning makes Minimax more efficient and is particularly useful when the game tree is large and decisions must be made quickly.

(d) Design an evaluation function and justify the factors considered.

Answer:

An evaluation function estimates how favourable a game state is for the AI when searching cannot continue to a terminal state.

For a real-time strategy game, a possible evaluation function is:

$$\begin{aligned} E(s) = {}& 5M + 3R + 4T \\ &+ 6B + 2I - 5H \end{aligned}$$

Where:

- M = military strength advantage.
- R = resource advantage.
- T = territory control advantage.
- B = base and building advantage.
- I = intelligence or reconnaissance advantage.
- H = threat or damage disadvantage.

The numerical weights are illustrative and should be tuned and tested for the particular game.

Justification of factors:

1. Military strength: Measures the relative strength of available units.
2. Resources: Evaluates the ability to build units and maintain operations.
3. Territory control: Rewards control of strategically important areas.
4. Base and buildings: Represents defensive strength and production capacity.
5. Intelligence: Rewards useful information about the opponent and the battlefield.
6. Threat and damage: Penalizes dangerous situations and significant losses.

The AI evaluates each state using these factors and prefers states with higher evaluation values. For the opponent's turn, Minimax assumes that the opponent will try to reduce the AI's advantage.

Conclusion: A good evaluation function considers the most important strategic factors and provides a reliable estimate of the likely outcome of a game state.

(e) Recommend a strategy for balancing optimality and real-time performance.

Answer:

The most suitable strategy is to combine Minimax, Alpha-Beta pruning, a heuristic evaluation function, and depth-limited search.

The reasons are:

1. Alpha-Beta pruning: Reduces unnecessary exploration of the game tree.
2. Depth-limited search: Restricts the search to a manageable number of moves.
3. Evaluation function: Estimates the quality of states that have not reached the end of the game.
4. Move ordering: Examines promising moves first to improve pruning effectiveness.
5. Time limits: Stops the search before the decision deadline.
6. Iterative deepening: Searches progressively greater depths and retains the best move from the last completed search.
7. Adaptive depth: Uses more computation when time and resources allow and less when rapid decisions are necessary.

For a complex real-time strategy game, additional techniques such as beam search or domain-specific action selection may be useful. Standard Minimax is most directly applicable when the game can be modelled as alternating decisions between two opposing players.